

LOS Lab Assignment 3

Part A — Basic grep

Exercise 1 — Search File Contents (using payload.sh)

Find the line containing wget.

```
grep "wget" payload.sh
```

Find the line containing nc.

```
grep "nc" payload.sh
```

Find the line containing chmod.

```
grep "chmod" payload.sh
```

Search for WGET ignoring case.

```
grep -i "WGET" payload.sh
```

Display the line numbers containing /tmp.

```
grep -n "/tmp" payload.sh
```

Count the number of lines containing /tmp.

```
grep -c "/tmp" payload.sh
```

Display only the matched word wget.

```
grep -o "wget" payload.sh
```

Search for nc as a whole word.

```
grep -w "nc" payload.sh
```

Exercise 2 — Compare Files

Search for wget in: payload.sh, update.sh, cleanup.sh

```
grep "wget" payload.sh update.sh cleanup.sh
```

Which file contains wget?

```
grep -l "wget" payload.sh update.sh cleanup.sh
```

Answer: payload.sh

Which files do not contain wget?

```
grep -L "wget" payload.sh update.sh cleanup.sh
```

Answer: update.sh and cleanup.sh

Repeat the search for /tmp.

```
grep "/tmp" payload.sh update.sh cleanup.sh
```

Which files contain /tmp?

```
grep -l "/tmp" payload.sh update.sh cleanup.sh
```

Use grep with multiple filenames (count per file).

```
grep -c "/tmp" payload.sh update.sh cleanup.sh
```

Part B — Security Log Analysis

Exercise 3 — Inverse Search (using system.log)

Display all lines that do not contain INFO.

```
grep -v "INFO" system.log
```

Display all lines that do not contain ERROR.

```
grep -v "ERROR" system.log
```

Display all lines except those containing WARNING.

```
grep -v "WARNING" system.log
```

Count the number of non-INFO entries.

```
grep -vc "INFO" system.log
```

Exercise 4 — Analyze system.log

Find all ERROR entries.

```
grep "ERROR" system.log
```

Find all WARNING entries.

```
grep "WARNING" system.log
```

Find the FAILED entry.

```
grep "FAILED" system.log
```

Count the number of ERROR entries.

```
grep -c "ERROR" system.log
```

Display line numbers containing WARNING.

```
grep -n "WARNING" system.log
```

Search for error irrespective of case.

```
grep -i "error" system.log
```

Determine whether the word Malware occurs, without displaying output.

```
grep -q "Malware" system.log  
echo $?
```

Answer: exit status 1 → "Malware" was not found in system.log.

Exercise 5 — Context Investigation

An ERROR event has been detected in system.log.

Display two lines after each ERROR.

```
grep -A 2 "ERROR" system.log
```

Display two lines before each ERROR.

```
grep -B 2 "ERROR" system.log
```

Display one line before and one line after each ERROR.

```
grep -C 1 "ERROR" system.log
```

Display two lines around FAILED.

```
grep -C 2 "FAILED" system.log
```

Why can examining events before and after an error be useful during incident investigation?

Examining events before and after an error helps investigators understand what happened immediately before and after the incident. It can reveal the cause of the error, related suspicious activity and what happened afterward.

Part C — egrep / grep -E

Exercise 6 — Search Multiple Security Events

Using system.log, find lines containing ERROR, WARNING, or FAILED in a single command.

```
grep -E "ERROR|WARNING|FAILED" system.log
```

Exercise 7 — Multiple Suspicious Commands

Using payload.sh, search for any of wget, curl, nc, chmod using a single extended regular expression.

```
grep -E "wget|curl|nc|chmod" payload.sh
```

Which patterns were found?

Answer: wget, nc, chmod

Which pattern was not found?

Answer: curl

How many lines contain at least one of these patterns?

```
grep -Ec "wget|curl|nc|chmod" payload.sh
```

Exercise 8 — Search Multiple Files

Which files contain at least one selected command?

```
egrep "wget|curl|nc|chmod" payload.sh update.sh cleanup.sh
```

Display only the filenames containing matches.

```
egrep -l "wget|curl|nc|chmod" payload.sh update.sh cleanup.sh
```

Display matching lines with line numbers.

```
egrep -n "wget|curl|nc|chmod" payload.sh update.sh cleanup.sh
```

Count the matching lines in each file.

```
egrep -c "wget|curl|nc|chmod" payload.sh update.sh cleanup.sh
```

Part D — patterns.txt

Exercise 9 — Pattern File Investigation

Search payload.sh using patterns.txt.

```
grep -f patterns.txt payload.sh
```

Search system.log using patterns.txt.

```
grep -f patterns.txt system.log
```

Search config.conf using patterns.txt.

```
grep -f patterns.txt config.conf
```

Search all .sh files using patterns.txt.

```
grep -f patterns.txt *.sh
```

Display only filenames containing at least one pattern from patterns.txt.

```
grep -lf patterns.txt *.sh
```

Display matching lines without printing filenames.

```
grep -hf patterns.txt *.sh
```

Which files contain indicators listed in patterns.txt?

```
grep -l -f patterns.txt *
```

Part E — Regex Using filelist.txt

Exercise 10 — Beginning and End of Line

Find filenames beginning with malware.

```
grep "^malware" filelist.txt
```

Find filenames beginning with script.

```
grep "^script" filelist.txt
```

Find filenames ending in .sh.

```
grep "\.sh$" filelist.txt
```

Find filenames ending in .py.

```
grep "\.py$" filelist.txt
```

Find filenames ending in .log.

```
grep "\.log$" filelist.txt
```

Find filenames beginning with p and ending in .sh.

```
grep "^p.*\.sh$" filelist.txt
```

Exercise 11 — Character Classes

Find malware1.sh and malware2.sh using one pattern.

```
grep "^malware[12]\.sh$" filelist.txt
```

Find filenames containing a digit.

```
grep "[0-9]" filelist.txt
```

Find filenames containing digits 1 or 2.

```
grep "[12]" filelist.txt
```

Find filenames beginning with m or p.

```
grep "^[mp]" filelist.txt
```

Find filenames beginning with s or u.

```
grep "^[su]" filelist.txt
```

Exercise 12 — Negated Character Classes

Find filenames that do not begin with m.

```
grep "^[^m]" filelist.txt
```

Find filenames that do not begin with m or p.

```
grep "^[^mp]" filelist.txt
```

Find malware filenames where the character following malware is not a digit.

```
grep "^malware[^0-9]" filelist.txt
```

Part F — File Operators

Exercise 13 — File Existence & Type Checks

Write a shell script that accepts a filename and determines whether it exists. Test with: payload.sh, report.txt, unknown.sh

```
#!/usr/bin/bash
read -p "Enter filename: " filename
if [ -e "$filename" ]
then
    echo "$filename exists"
else
    echo "$filename does not exist"
fi
```

Run: chmod +x check_file.sh then ./check_file.sh

Write a script that accepts a path and determines whether it is a regular file, a directory, or neither. Test with: payload.sh, system.log, quarantine, backup, unknown

```
#!/usr/bin/bash
read -p "Enter path: " path
if [ -f "$path" ]; then
    echo "$path is a regular file"
elif [ -d "$path" ]; then
    echo "$path is a directory"
else
    echo "$path is neither"
fi
```

Run: `chmod +x check_path.sh` then `./check_path.sh`

Exercise 15 — Permission Reporting

Write a script that reports whether a selected file is readable, writable, and executable. Test with: `payload.sh`, `update.sh`, `report.txt`, `config.conf`

```
#!/usr/bin/bash
read -p "Enter filename: " filename
if [ -r "$filename" ]; then echo "$filename is readable"; fi
if [ -w "$filename" ]; then echo "$filename is writable"; fi
if [ -x "$filename" ]; then
    echo "$filename is executable"
else
    echo "$filename is not executable"
fi
```

Run: `chmod +x filePermissions.sh` then `./filePermissions.sh`

Exercise 16 — Empty vs Non-empty

Compare `payload.sh` and `empty.txt`: write a script that determines whether each file contains data.

```
#!/usr/bin/bash
read -p "Enter filename: " filename
if [ -s "$filename" ]; then
    echo "$filename contains data"
else
    echo "$filename is empty"
fi
```

Run: `chmod +x fileData.sh` then `./fileData.sh`

Part G — File Validation Using []

Exercise 17 — File Validation

Write a script that accepts a filename and checks: does it exist, is it a regular file, is it readable, is it non-empty?

```
#!/usr/bin/bash
read -p "Enter filename: " filename
if [ -e "$filename" ]; then echo "$filename exists"; fi
if [ -f "$filename" ]; then echo "$filename is a regular file"; fi
if [ -r "$filename" ]; then echo "$filename is readable"; fi
if [ -s "$filename" ]; then echo "$filename is non-empty"; fi
```

Run: `chmod +x validate.sh` then `./validate.sh`

Exercise 18 — [] + grep

Chain the checks above, then search for "wget" and report whether the selected indicator was detected. Test using both `payload.sh` and `update.sh`

```
#!/usr/bin/bash
read -p "Enter filename: " filename
if [ -e "$filename" ] && [ -f "$filename" ] && [ -r "$filename" ] && [ -s "$filename" ]; then
    echo "$filename exists"
    echo "$filename is a regular file"
    echo "$filename is readable"
    echo "$filename is non-empty"
    if grep -q "wget" "$filename"; then
        echo "Selected indicator detected - investigate further"
    else
        echo "Selected indicator not detected"
    fi
fi
```

Run: `chmod +x validation.sh` then `./validation.sh`

Part H — Repeating Tests with [[]]

Exercise 19 — Repeat File Tests Using [[]]

Repeat Exercise 17/18 using `[[ ]]` instead of `[ ]`.

```
#!/usr/bin/bash
read -p "Enter filename: " filename
if [[ -e "$filename" ]] && [[ -f "$filename" ]] && [[ -r "$filename" ]] && [[ -s "$filename" ]]; then
    echo "$filename exists"
    echo "$filename is a regular file"
    echo "$filename is readable"
    echo "$filename is non-empty"
    if grep -q "wget" "$filename"; then
        echo "Selected indicator detected - investigate further"
    fi
fi
```

Run: `chmod +x repeatedVal.sh` then `./repeatedVal.sh`

Exercise 20 — Regex Using [[]]

Write a shell script that checks whether the filename ends with `.sh`, using `[[ "$filename" =~ regex ]]`. Test: `payload.sh`, `scanner.py`, `report.txt`

```
#!/usr/bin/bash
read -p "Enter filename: " filename
if [[ "$filename" =~ \.sh$ ]]; then
    echo "Shell script identified for inspection"
else
    echo "File does not have a .sh extension"
fi
```

Run: `chmod +x regex.sh` then `./regex.sh`

Exercise 21 — Multiple Extensions Using `[]`

Extend the previous script to identify `.sh`, `.py`, and `.pl` files without classifying them as malicious. Test: `payload.sh`, `scanner.py`, `report.txt`, `config.conf`

```
#!/usr/bin/bash
read -p "Enter filename: " filename
if [[ "$filename" =~ \.(sh|py|pl)$ ]]; then
    echo "Script file - inspect if required"
else
    echo "File does not match a supported extension"
fi
```

Run: `chmod +x ex21.sh` then `./ex21.sh`

Part I — `grep` + File Operators + Constructs

Exercise 22 — Executable File Content Analysis

Write a script that accepts a filename, checks existence, checks it's a regular file, checks it's executable, and if so searches for `wget`. Test: `payload.sh`, `update.sh`, `report.txt`

```
#!/usr/bin/bash
read -p "Enter filename: " filename
if [ -e "$filename" ]; then
    echo "$filename exists"
    if [ -f "$filename" ]; then
        echo "$filename is a regular file"
        if [ -x "$filename" ]; then
            echo "$filename is executable"
            if grep -q "wget" "$filename"; then
                echo "wget found"
            else
                echo "wget not found"
            fi
        else
            echo "$filename is not executable"
        fi
    fi
fi
```

Run: `chmod +x execute.sh` then `./execute.sh`

Exercise 23 — `egrep` + File Operators

Only perform content analysis when the file is regular, readable, and non-empty; then use `egrep/grep -E` to search for `wget|curl|nc|chmod`.

```
#!/usr/bin/bash
read -p "Enter filename: " filename
if [ -f "$filename" ] && [ -r "$filename" ] && [ -s "$filename" ]; then
    if grep -Eq "wget|curl|nc|chmod" "$filename"; then
        echo "One or more selected indicators detected"
    else

```

```
        echo "Selected indicators not detected"
    fi
fi
```

Run: `chmod +x reg.sh` then `./reg.sh`

Part J — Configuration File Investigation

Exercise 24 — Sensitive Information Search

Use `config.conf`. First verify it exists, is a regular file, is readable, and is non-empty.

```
[ -e config.conf ] && [ -f config.conf ] && [ -r config.conf ] && [ -s config.conf ] && echo "config.conf OK"
```

Then use `egrep` to search for `password|username` and display the matching lines.

```
egrep "password|username" config.conf
```

Why should configuration files containing authentication information receive additional protection?

Configuration files may contain sensitive authentication information such as usernames and passwords. Unauthorized access to these files could allow an attacker to obtain credentials and gain access to systems or services.

Part K — Suspicious File Triage

Exercise 25 — `file_check.sh`

A single Bash program that asks for a filename and reports: whether the path exists, is a regular file, is a directory; and if it's a regular file — whether it's readable, writable, executable, and empty/non-empty. If readable and non-empty, it searches for `wget` (and for `wget|curl|nc|chmod` using `grep -E`), using `$?` to check each search result, and uses `[[ ]]` with `regex` to detect a `.sh` extension. It finishes by printing a summary.

```
#!/usr/bin/bash

read -p "Enter filename: " filename

echo
echo "----- FILE INVESTIGATION -----"
echo

# Exists?
if [ -e "$filename" ]; then
    exists="Yes"
else
    exists="No"
fi

# Regular file?
if [ -f "$filename" ]; then
    regular="Yes"
else
    regular="No"
fi
```

```

# Directory?
if [ -d "$filename" ]; then
    directory="Yes"
else
    directory="No"
fi

# Readable, writable, executable, non-empty (only if it is a regular file)
if [ "$regular" = "Yes" ]; then
    [ -r "$filename" ] && readable="Yes" || readable="No"
    [ -w "$filename" ] && writable="Yes" || writable="No"
    [ -x "$filename" ] && executable="Yes" || executable="No"
    [ -s "$filename" ] && nonempty="Yes" || nonempty="No"
else
    readable="N/A"; writable="N/A"; executable="N/A"; nonempty="N/A"
fi

# .sh extension check using [[ ]] with regex
if [[ "$filename" =~ \.sh$ ]]; then
    shellscrip="Yes"
else
    shellscrip="No"
fi

echo "Exists      : $exists"
echo "Regular File : $regular"
echo "Directory   : $directory"
echo "Readable    : $readable"
echo "Writable     : $writable"
echo "Executable   : $executable"
echo "Non-empty    : $nonempty"
echo "Shell Script : $shellscrip"
echo

# Content search only if readable and non-empty
if [ "$readable" = "Yes" ] && [ "$nonempty" = "Yes" ]; then
    if grep -q "wget" "$filename"; then
        echo "wget found"
    else
        echo "wget not found"
    fi

    if grep -Eq "wget|curl|nc|chmod" "$filename"; then
        echo "grep -E indicators : Match found"
    else
        echo "grep -E indicators : No match"
    fi
fi

```

Run: chmod +x file_check.sh then ./file_check.sh

Example output for payload.sh:

Enter filename: payload.sh

----- FILE INVESTIGATION -----

Exists : Yes
Regular File : Yes
Directory : No
Readable : Yes
Writable : Yes
Executable : Yes
Non-empty : Yes
Shell Script : Yes

wget found
grep -E indicators : Match found